Database Project Final Report

Eddy Hu Jaik Tom

June 28, 2026

1 Project Overview

This project implements a full-stack Air Ticket Reservation System using MySQL, Flask, and HTML/CSS/JavaScript. The system supports two user roles: Customers and Airline Staff. Customers can register, log in, search for flights, purchase tickets, manage reservations, and rate completed flights. Airline Staff can manage flights and airplanes, update flight status, and view reports and customer feedback. The application follows the relational database design developed in Parts 1 and 2 of the course project and provides a web interface for interacting with the database.

- Database: MySQL
- Backend: Flask (Python)
- Frontend: HTML/CSS/JavaScript
- Version Control: Git/GitHub

2 Use Cases

The application is organized around three groups of use cases: public (not logged in), customer, and airline staff. Each use case is implemented by a Flask route in `app.py` and executes one or more parameterized SQL queries against the `airline_db` database. The list below summarizes each use case and the queries it runs; the companion document *Use Cases and Queries* gives the full SQL.

2.1 Public Use Cases (no login required)

- **View public home page** (`/`) — entry point with the flight-search box and register/login links. No query.
- **Search for available flights** (`/search`) — one-way or round-trip search by source, destination, and date. Joins `Flight` with `Airport` (twice, for departure and arrival) and matches on either city or airport name and the departure date; the round-trip option re-runs the same query with source and destination swapped.

2.2 Registration and Authentication

- **Customer registration** (`/register/customer`) — checks `Customer` for an existing email, then `INSERT`s a new customer with the password hashed via `MD5()`.

- **Staff registration** (`/register/staff`) — verifies the username is free and the airline exists, `INSERT`s into `Airline_Staff` (MD5 password), and `INSERT`s one row per phone number into `Staff_Phone`.
- **Login** (`/login`) — role-aware authentication that checks `Airline_Staff` or `Customer` for a matching username/email and `MD5()` password, then starts a session.
- **Logout** (`/logout`) — clears the session. No query.

2.3 Customer Use Cases

- **Customer home** (`/customer`) — lists the customer's upcoming flights by joining `Ticket` to `Flight` and keeping rows with `departure_datetime >= NOW()`.
- **View my flights** (`/customer/my-flights`) — the customer's tickets with optional filters (scope future/past, source, destination, date range) added dynamically to the `Ticket` ⋈ `Flight` ⋈ `Airport` query.
- **Purchase ticket** (`/customer/purchase`) — checks seat availability by comparing `Airplane.num_seats` against a `COUNT` of booked `Ticket`s, and if seats remain `INSERT`s a new ticket (with payment fields and `purchase_datetime = NOW()`).
- **Rate a flight** (`/customer/rate`) — verifies the customer actually took the flight (lookup in `Ticket`), then `INSERT`s into `Review` with `ON DUPLICATE KEY UPDATE`; the GET view lists past flights eligible to rate.

2.4 Airline Staff Use Cases

- **Staff home** (`/staff`) — the airline's flights departing within the next 30 days from `Flight`.
- **View / search flights** (`/staff/flights`) — lists the airline's flights filtered by scope (future/past/all/range) and can drill into the passengers of a chosen flight by joining `Ticket` with `Customer`.
- **Create flight** (`/staff/flights/create`) — checks for a duplicate flight, then `INSERT`s into `Flight`; the form is populated from `Airport` and the airline's `Airplane` rows.
- **Change flight status** (`/staff/flights/status`) — `UPDATE`s `Flight.status` (e.g. on-time / delayed).
- **Add airplane** (`/staff/airplanes`) — checks for a duplicate ID, then `INSERT`s into `Airplane`; also lists the airline's existing fleet.
- **View ratings** (`/staff/ratings`) — per-flight average rating (`AVG` over `Review`) plus every comment, joined to `Customer` for the reviewer's name.
- **Sales reports** (`/staff/reports`) — total ticket count and revenue for a date range, plus a month-by-month breakdown using `DATE_FORMAT` and `GROUP BY` over `Ticket` ⋈ `Flight`.
- **Flight detail** (`/staff/flights/detail`) — full detail for one flight: flight info, the airplane (`LEFT JOIN Airplane`), and the booked passengers (`Ticket` ⋈ `Customer`).

3 Project Structure

```
database_project/
 app.py              # Main Flask app
 db.py               # get_connection() helper
 requirements.txt    # Python dependencies
 static/
    css/
       style.css
    img/
        background.png
 templates/
     base.html             # Master layout extended by every page
     _macros.html          # Reusable macros
     home.html             # Public landing page, search box
     search.html           # Public flight-search results
     register.html
     register_customer.html# Customer sign-up
     register_staff.html   # Airline-staff sign-up
     login.html
     goodbye.html
     customer/
        home.html         # Customer dashboard
        my_flights.html   # Purchased flights
        purchase.html     # Ticket-purchase
        rate.html         # Rate & comment on past flights
     staff/
         home.html         # Staff dashboard
         view_flights.html # Flight browser
         create_flight.html# Create flight form
         change_status.html# Update flight status
         add_airplane.html # Add airplane to the fleet
         flight_detail.html# Single-flight detail
         ratings.html      # Average ratings
         reports.html      # Ticket-sales reports
```

4 Team Contributions

Member	Responsibilities
Eddy Hu	View Public Info, Customer register, Customer login, Customer use cases, testing
Jaik Tom	Airline Staff register, Airline Staff log in, Airline Staff Use cases, testing
Claude	Frontend, CSS stylesheet, templates

5 Project Timeline

Week	Milestone
1	Requirements analysis and ER design
2	Relational Schema and Database Implementation
3	Backend development and integration
4	Submission

6 GitHub Activity

- Feature branches (customer & staff) for major components
- Pull Requests used for code review
- Frequent commits throughout development
- Required approval before merge

Repository Statistics

- Total commits: 18
- Pull Requests: 2
- Contributors: 2

7 Git Log

```
33cb947 Merge pull request #3 from syttpz/staff
7751991 (origin/staff) Merge branch 'main' into staff
bb6d84a Conflict resolutions
32d79a9 Maintenance Changes
73fbea2 Merge pull request #1 from syttpz/customer
b12bbe2 Update: Demo Removal and Staff Airline Creation Revoked
1c746de Feat: Full Airline Staff Implementation
cc4b23e (origin/customer, customer) fixed expiration date placeholder
79c9cfe fixed expiration date
6180d01 fixed typo, bugs
e216107 customer rate implemented
d6f0079 customer_ticket_purchase
ab37655 implemented customer_home, customer_my_flights
e539351 Checks registered customer, customer already exist, implemented login feature
6be3ca0 implemented customer registry
a0e6dbd implemented search
52c74ee init
```

8 Testing

- User authentication
- Flight search
- Ticket booking
- Ticket cancellation
- Payment validation
- SQL constraint verification